

Company Brain

The Architecture of General Company Intelligence

Oleksandr Vityaz

Corezoid Inc., Dnipro, Ukraine

alexander.vityaz@gmail.com

corezoid.com · simulator.company

[NotebookLM](#)

Abstract

This paper proposes Company Brain—an architectural framework for the cognitive organ of an enterprise, emerging when a digital core, a digital twin, meta-regulation, and human goal-setting converge into a single regulatory loop. The theoretical foundation is the Conant–Ashby theorem: every good regulator must be a model of the system it regulates. We argue that meeting this requirement demands not an analytical but a geometric model of the organisation—a computable representation of actors, roles, states, constraints, and transitions. The operational form of such a model is the Actor Graph (AG), and its concrete instantiation for a given organisation is the Big Graph of Company Context (BGCC). We describe a four-phase evolutionary path from a Fragmented Company through a Digital Core and a Digital Twin to an Autonomous Enterprise. For each phase, the architectural ceiling, transition mechanism, and human role are defined. Artificial intelligence is positioned as a local computational framework inside the AG macro-framework, not as a sovereign regulator. A Hybrid Intelligence protocol is presented as a formal transaction loop between the AI subgraph and the human within AG. We conclude with implications for practice and an explicit discussion of current limitations.

Keywords: Company Brain, Actor Graph, digital twin of organisation, Conant–Ashby theorem, Viable System Model, management debt, autonomous enterprise, hybrid intelligence, organisational cybernetics, enterprise architecture.

1 From Metaphor to Architecture

At a certain point, a growing company stops running into a shortage of data and starts running into a shortage of form—the form in which it can understand itself. There are more signals, more systems, more decisions—and less control. This is where Stafford Beer’s metaphor of the “brain of the firm” [5] stops being a beautiful analogy and becomes an engineering problem.

The logic of this problem is unforgiving. The Conant–Ashby theorem [8] requires that a good regulator contain a model of the system it regulates. For business, that means one simple thing: until a company is represented to itself in the form of a working model, it does not regulate its own trajectory; it merely reacts to the consequences of events that have already happened. Reports, dashboards, KPIs, and retrospective analytics help you see the traces of the past, but they do not create an organ for governing the future.

So the main question is not, “How many AI tools has the company deployed?” The main question is this: **“In what form does the company exist for itself as an object of computation, change, and control?”**

Analytical descriptions are not enough for that. Maxwell’s equations describe why radio waves exist, but you cannot build a receiver from them: you need a schematic—a concrete topology of components, connections, tolerances, and admissible states. Einstein’s $E = mc^2$ tells you that

a nuclear reaction is possible, but between that formula and a working reactor lie thousands of engineering drawings in which every control rod, every cooling loop, and every sensor has a place, a role, and a range of permitted behaviour. The same gap separates an analytical model of a company (KPIs, dashboards, statistical summaries) from a **geometric model** in the engineering sense: a computable representation of actors, roles, resources, relationships, states, constraints, and transitions—closer to a CAD assembly than to a system of equations. An analytical model tells the company *what is happening*; a geometric model shows it *how it is constructed* and therefore *what can be changed*. Until the company possesses such a model, it can observe itself but it cannot design itself.

When that geometric model begins to synchronise continuously with the company’s actual state, it becomes a **digital twin** [10]. A digital twin is not a separate entity sitting next to the model; it is a stronger mode of the same model: the same geometry, now brought into alignment with reality. The same model may exist in several versions at once: as the current state, a historical slice, a target configuration, or a scenario branch. A company that has brought its geometric model into DTO mode becomes a **Smart Company**.

The operational form of this living geometry is the **Actor Graph (AG)**. AG is the language and the processing mechanism of the model [21]. **BGCC**—the Big Graph of Company Context—is the concrete world of a given company already expressed in AG: its own geometry of roles, processes, constraints, transition rights, and active contexts. In other words, AG is the grammar and the machine; BGCC is the concrete company assembled on that grammar.

2 Motivating Example: One Incident, Four Architectures

Take a standard case. A customer writes in chat, calls the contact centre, and files a ticket in the app: the money has been charged, but the service has not been activated.

In a *fragmented company*, that single signal turns into three different events. The contact centre sees a customer complaint, billing sees a payment, support sees an incident, and the product team sees a likely integration error. Clarifications begin. Messages are forwarded. Escalations start. Ownership gets disputed. Time is spent not on solving the problem, but on figuring out who exactly is supposed to solve it.

In a company with a *digital core*, those three inputs are normalised into one pragmatic event class. The system understands that for this type of failure the set of admissible actions is the same: verify the charge, verify the activation, either restore the service or issue a refund, or escalate the case into an exception loop.

In a *Smart Company*, the same case already lives inside the organisation’s digital twin. The system sees not just an incident, but its place in the overall geometry of the business: which actors are involved, which SLAs are being violated, where duplicates already exist, what the cost of delay is, and what will happen if the route changes.

In an *Autonomous Enterprise*, such a case will, in most instances, require no human in the operating loop at all. Standard event classes are processed automatically, and the human receives only what cannot be suppressed or delegated by the architecture itself: changes in goals, policy, the boundaries of the admissible, or genuinely algedonic signals—alerts that by definition must bypass the normal filters [7].

The same incident. Four different architectures. The difference lies not in the number of dashboards and not in the power of the LLM, but in the quality of the company’s internal model.

3 Four Phases of Evolution

The evolutionary path described here follows a four-phase model [22], summarised in Fig. 1.

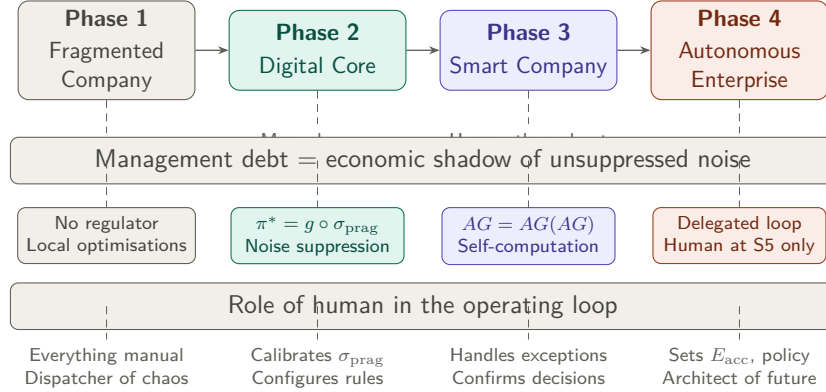


Figure 1: Four-phase evolution model. Each phase has an architectural ceiling that forces the transition to the next. Management debt is the economic accounting of unsuppressed organisational noise. We write $AG = AG(AG)$ as shorthand for Eq. (2).

3.1 Phase 1: Fragmented Company

The initial state of a complex organisation is fragmentation. Systems appear reactively, in response to the needs of individual departments. Logic is spread across CRM, ERP, chats, spreadsheets, people, local scripts, and implicit agreements. The formal org chart almost never matches the real structure of decision-making.

In this state, the company does not possess a unified regulator. Instead, it has a set of local regulators, each governing its own fragment of reality. At the level of an individual function, this can still look like efficiency. At the level of the company, it produces noise: duplicated decisions, blurred ownership, repeated discussions, meaningless escalations, reopening of issues that were already resolved, and endless clarification. This is where **management debt** appears—the economic shadow of unsuppressed organisational noise [9, 23].

Operationally, management debt accrues whenever a distinction that σ_{prag} fails to suppress forces a human to perform coordination work that the regulator should have handled: forwarding a message to find the owner, re-entering data across systems, convening a meeting to reconcile two contradictory statuses, or reopening a decision that was already made elsewhere. Each such instance carries a measurable cost in time, attention, and error rate. In aggregate, these micro-costs constitute the debt balance that grows silently until it dominates the organisation’s coordination budget. A fuller treatment—including double-entry attribution of materialised debts to actor accounts—is given in [23]; here we note that management debt serves as the accounting instrument that makes noise economically visible and therefore manageable.

Phase 1 is dangerous not simply because the noise level is high, but because noise grows without control. Every new system increases the variety of inputs, interfaces, and points of misalignment, but creates no common mechanism for suppressing pragmatically irrelevant distinctions [2]. Local efficiency can improve inside Phase 1, while global regulation cannot. At some point, coordination costs start growing faster than the benefit of the next automation. That is the point at which the company either builds a common regulator or keeps paying for noise.

3.2 Phase 2: Digital Core

The move to the second phase begins when the company builds a unified digital core. Its task is to take over state transitions, event routing, and computational discipline. This is the first point at which a common regulator appears, rather than merely a set of connected systems.

Architecturally, that transition is expressed by the formula [25]

$$\pi^* = g \circ \sigma_{\text{prag}} \quad (1)$$

First, σ_{prag} suppresses distinctions that do not change the set of admissible actions: it normalises events, deduplicates signals, and maps different inputs into the same pragmatic class of situation [4]. Then g selects the action, route, owner, deadline, risk level, and need for escalation—not for the raw signal, but for its pragmatic class. In business terms, this means something simple: the company stops spending cognitive and organisational power on noise and begins regulating only the variety that actually affects the outcome. Where such separation does not exist, management debt grows as the economic shadow of unsuppressed organisational noise.

In this architecture, legacy systems do not have to disappear. They are encapsulated and turned into controlled black boxes, connected to the common regulator through programmatic interfaces and SLAs [12, 17]. The company does not destroy the existing landscape; it subordinates it to a single logic of transitions.

But Phase 2 has a ceiling. The digital core already knows how to suppress noise and route the flow, yet the projection σ_{prag} is still largely calibrated by hand. As complexity grows, classification errors begin to accumulate: some meaningful signals are suppressed, some duplicates are detected too late, and the rules stop keeping up with reality. The company is already managing the flow, but it still does not possess a full model of the object being managed. That is what forces the move to Phase 3.

3.3 Phase 3: Digital Twin / Smart Company

The third phase begins where the company builds a full geometric model of the organisation and brings it into digital twin mode. Here the regulator no longer merely manages events; it contains an explicit, computable model of the company itself.

This is also the phase at which the second principle of the architecture is activated [24]:

$$AG = (\lambda AG. AG(AG)) (\lambda AG. AG(AG)) \quad (2)$$

We write $AG = AG(AG)$ as shorthand throughout the paper.

In pure untyped λ -calculus [3], the combinator $(\lambda x. x x) (\lambda x. x x)$ diverges: it has no normal form. Why, then, does the same notation express a well-founded principle when applied to Actor Graphs?

The answer lies in three engineering constraints that are absent in the pure calculus but constitutive for AG:

1. **Transactional tact.** Every rewriting step in AG is a discrete, committed transaction with a well-defined pre-state and post-state. The graph advances one transaction at a time; there is no unbounded β -reduction chain within a single step.
2. **Invariant guards.** The S3 layer (see below) enforces hard invariants that reject any transaction violating structural or policy constraints. These guards act as the productivity conditions familiar from coinductive definitions [15]: they guarantee that every step produces an observable, finite change.

3. **Finite state space per step.** At any given transaction, the set of active nodes, edges, and admissible transitions is finite. Self-application therefore does not generate an infinite descent but a bounded reconfiguration of a finite structure.

Under these constraints, the self-application in Eq. (2) is not divergent recursion but *guarded corecursion*: an indefinitely productive process that yields a well-defined state after every step. Every transaction changes the graph itself—node states, the composition of edges, active contexts, transition rights, model versions, and computational routes. So the regulator is not merely acting inside the graph; it is continuously rewriting its own geometry. In this sense, AG is both the medium of computation and the object being changed by that computation. Self-computation here is not a metaphor. It follows directly from the fact that the graph describes a hierarchy of frameworks that includes the graph itself.

That is why a Smart Company gets more than a process layer; it gets a computable body of its own. It can see the organisation’s current topology, compare versions, model changes, test hypotheses, and validate new configurations before rolling them into reality.

Meta-regulation and its mapping to AG. To keep such a digital twin from collapsing into endless self-reconfiguration, meta-regulation is required. In Beer’s terms [6, 7], this is the S3, S4, and S5 stack. Each level maps onto concrete architectural elements inside the Actor Graph:

- **S3 — Hard operational invariants.** Implemented as *invariant nodes* and *guard edges* in AG. Before any transaction is committed, the S3 subgraph evaluates it against a set of inviolable constraints: regulatory limits, security policies, data-integrity rules, and financial controls. A transaction that fails an S3 check is rejected or routed into an exception channel. S3 is the mechanism that makes self-computation productive rather than divergent.
- **S4 — Adaptation to the environment.** Implemented as a dedicated *adaptation sub-graph* that observes both the external environment (market signals, competitor actions, regulatory changes) and the internal state of the twin (drift between model and reality, classification error rates, SLA violations). S4’s output is a set of proposed modifications: recalibration of σ_{prag} parameters, adjustment of routing rules, revision of risk thresholds, or topology changes such as adding, removing, or rewiring actor nodes. These proposals are themselves transactions subject to S3 invariants.
- **S5 — Goals, policy, identity, and the boundaries of the admissible.** Implemented as a set of *policy nodes* that define E_{acc} , the identity constraints of the organisation, and the meta-rules governing what S4 is allowed to adapt. S5 is the only level at which the human is structurally required: the right to alter the criteria of admissibility, to redefine the organisation’s purpose, or to override the entire regulatory stack cannot be delegated without dissolving the distinction between the organisation and its regulator.

That is what allows the digital twin not merely to reflect reality, but to hold the organisation inside a computable and governable loop.

This architecture already exists beyond theory. In one engineering instantiation, the regulator’s compute layer is implemented in Corezoid, while the design and processing environment of the digital twin is implemented in simulator.company [21]. That does not make the concept product-dependent; it simply shows that what is described here is an engineering proof of viability rather than pure speculation.

But Phase 3 has a ceiling as well. Even with a complete digital twin, the human often remains part of the operating loop: handling exceptions, confirming non-standard decisions, adjusting routes, and recalibrating the parameters of σ_{prag} . The regulator’s throughput is still bounded by human throughput. That is the structural ceiling of the Smart Company.

3.4 Phase 4: Autonomous Enterprise

The autonomous enterprise does not begin where “AI does everything.” It begins where the human exits the routine operating loop and remains at the level of goals, policy, and boundaries.

At this phase, the regulator takes over the normalisation of the inbound flow, deduplication, routing, owner assignment, prioritisation, calendar and SLA discipline, bundling of similar actions, handling of standard exceptions, and most routine operational decisions. This is not a slogan about autonomy. It is a formalised loop.

Crucially, the boundary of autonomy is defined not by management rhetoric, but by the topology of AG itself. The graph must explicitly define auto-commit nodes, human-only nodes, hard invariants, bypass channels for algedonic signals, and human rights to alter the very criteria of admissibility. That is what removes the main fear around autonomy: Autonomous Enterprise is not the elimination of the human. It is the formal redistribution of the human role. The human stops being a dispatcher of the flow and becomes the architect of the admissible future.

At least four classes of decisions remain with the human: defining E_{acc} , the space of acceptable outcomes; handling algedonic signals; revising pragmatic equivalence itself (that is, changing what σ_{prag} treats as equivalent); and changing policy, identity, and system boundaries at the S5 level. The transition from Phase 3 to Phase 4 is the most dramatic of all: it means not just another level of automation, but a new distribution of cognitive labour between the human and the regulator.

4 What Company Brain Is

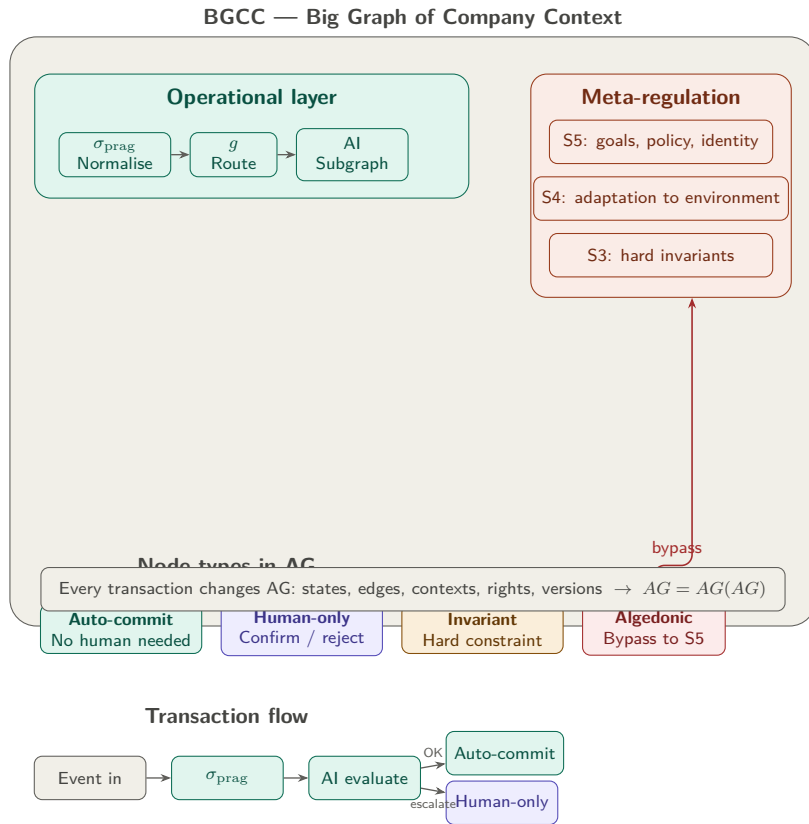


Figure 2: AG topology within BGCC. The operational layer implements π^* ; meta-regulation (S3/S4/S5) governs the digital twin. Four node types define the boundary of autonomy. Algedonic signals bypass all layers to reach S5. We write $AG = AG(AG)$ as shorthand for Eq. (2).

Company Brain is not another analytics layer laid on top of existing systems, and not an interface to corporate data. It is the company’s cognitive organ, emerging when the digital core, the digital twin, meta-regulation, and human goal-setting converge into a single regulatory loop.

The foundational layer of that organ is BGCC, the Big Graph of Company Context (Fig. 2). It holds not only the question “who is connected to whom?” but the more important question: “what is even possible, admissible, and computable here?” BGCC contains actors, roles, constraints, escalation routes, transition rights, active contexts, model versions, and traces of transactions that have already taken place.

Artificial intelligence inside this architecture occupies an important but subordinate place. LLMs, classifiers, predictive models, and other AI components are not the company’s sovereign mind. They are local computational frameworks inside a broader macro-framework [11, 18]. They can extract semantics, detect patterns, propose courses of action, and accelerate the processing of large volumes of signals. But by themselves they are not a regulator, because they do not contain the company’s full living model. Plasticity belongs not to any single model, but to the AG architecture as a whole.

5 How Hybrid Intelligence Works

The term **Hybrid Intelligence** makes sense only if it describes not “human–AI synergy” in general, but a concrete transaction protocol inside AG (Fig. 3).

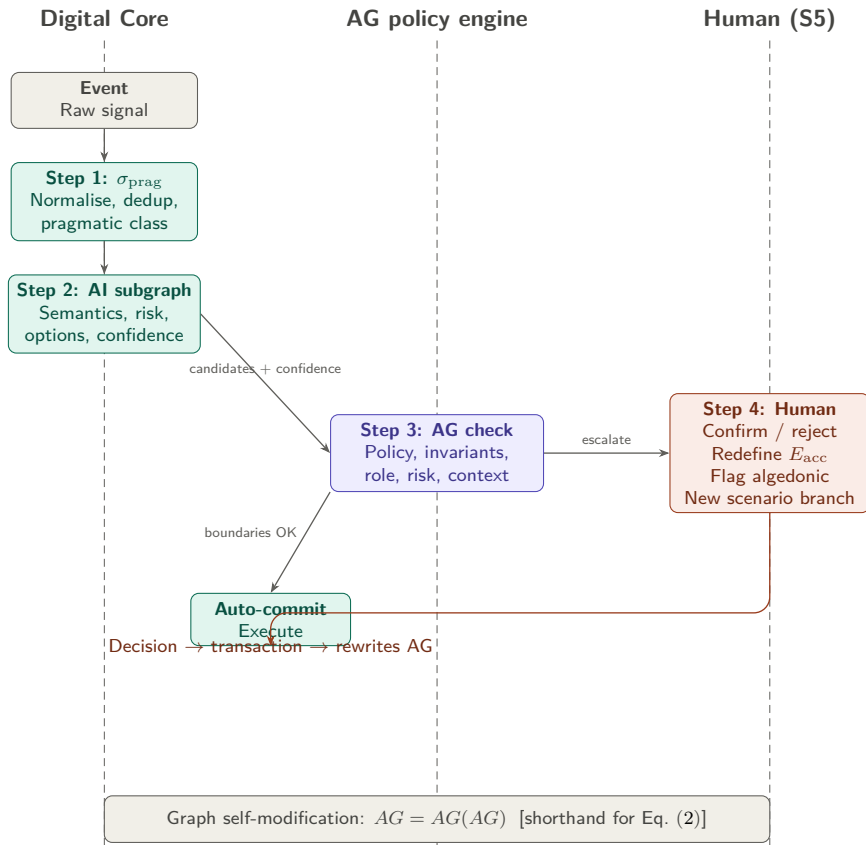


Figure 3: Hybrid Intelligence protocol. Events flow through σ_{prag} and AI evaluation before AG checks policy boundaries. The decision—whether auto-committed or human-confirmed—becomes a transaction that rewrites the graph.

In its minimal form, that protocol looks like this.

First, an event enters the digital core and passes through σ_{prag} : normalisation, deduplication, contextual linking, and assignment to a pragmatic class.

Then the AI subgraph extracts semantics, searches for historical analogues, assesses risk, and produces admissible courses of action together with a confidence profile.

After that, AG compares the proposed options against policy, invariants, actor role, risk level, and current context. If the situation class is fully described and the boundaries are respected, the decision is executed automatically. If not, the graph routes the event into a human-only node.

Finally, the human performs a strictly defined function: confirms, rejects, redefines E_{acc} , marks a signal as algedonic, shifts the boundary of a pragmatic class, or creates a new scenario branch. And that decision itself becomes a transaction that rewrites the graph.

The essence of hybrid intelligence lies in that final step. The human and AI do not merely stand next to one another; they are embedded in a common geometry of transitions. AI accelerates distinction and proposal. The human holds policy, meaning, responsibility, and the right to alter the structure of the admissible itself. AG turns that interaction from a vague “synergy” into a formal coordination loop.

6 Implications for Practice

The architecture described above has several concrete implications for how companies are built, acquired, and managed.

Integration and M&A. A company that possesses BGCC can absorb a new product, team, or acquisition by expressing it as a subgraph and connecting it to the existing topology through typed edges: shared actors, common constraints, unified event classes. The cost of integration is then a function of graph compatibility, not of the number of meetings required to “align cultures.” This does not eliminate the human work of integration, but it makes the structural part of that work computable and testable before it is executed.

Organisational hypothesis testing. In Phase 3 and beyond, the digital twin allows the company to branch its own model, apply a proposed reorganisation to the branch, simulate the flow of events through the new topology, and measure the predicted effect on throughput, coordination cost, and SLA compliance before committing the change. This is organisational design treated as engineering, not as politics.

Management debt as a metric. Once management debt is operationally defined (Section 3), it becomes a leading indicator. A rising debt balance in a particular subgraph signals that σ_{prag} is miscalibrated or that the topology has drifted from reality. The debt can be decomposed by actor, by process, and by event class, giving the organisation a diagnostic instrument that is finer-grained than any dashboard and more actionable than any retrospective.

The human role. In the old architecture, the human is a dispatcher compensating for the incompleteness of the system. In the new architecture, the human is the source of goals, policy, and meaning. Not an operator of chaos, but the architect of the admissible future. This is not a reduction of the human role; it is a concentration of it on the decisions that only humans can make: what the company should be, what it should not do, and what counts as an acceptable outcome.

7 Limitations and Future Work

This paper presents an architectural position. Several important questions remain open.

Scalability of BGCC synchronisation. Maintaining a digital twin in continuous alignment with a large organisation’s actual state is a non-trivial computational and organisational problem. The paper does not provide complexity bounds for twin synchronisation, nor does it address the latency and consistency trade-offs that arise when thousands of actors generate concurrent state changes. Formal analysis of the synchronisation protocol—including convergence guarantees under partial observability—is a necessary next step.

The bootstrapping problem. How does a Phase 1 company begin constructing its Actor Graph without already possessing one? The four-phase model describes the evolutionary sequence but does not prescribe a concrete onboarding methodology. In practice, bootstrapping likely begins with a partial graph covering one critical process or one high-noise subunit, expanding incrementally. Formalising this incremental construction and proving that partial graphs compose correctly under the AG axioms is an open problem.

Calibration of σ_{prag} . The factorisation $\pi^* = g \circ \sigma_{\text{prag}}$ assumes that pragmatic equivalence classes can be defined and maintained. In practice, σ_{prag} must be continuously recalibrated as the environment changes. The question of who validates that calibration—and how miscalibration is detected before it produces significant management debt—is the *quis custodiet* problem of this architecture. Partial answers exist (S4 observes drift; S5 can override), but a formal feedback loop with provable error bounds has not yet been constructed.

Empirical validation. The claims in this paper are architectural, not empirical. The four-phase model, the management debt metric, and the hybrid intelligence protocol have been instantiated in one engineering platform (Corezoid / simulator.company), but no controlled empirical study has yet been conducted to measure their effect on coordination cost, decision latency, or organisational adaptability across multiple firms. Such validation is essential for the framework to move from a plausible architecture to a tested one.

Formal semantics of AG. While the self-computation formula $AG = AG(AG)$ has been given an engineering interpretation grounded in transactional tact, invariant guards, and finite state space, a full denotational or coalgebraic semantics [15] of Actor Graphs—one that would allow formal verification of liveness, safety, and fairness properties—remains to be developed.

Conclusion

Company Brain is neither a metaphor nor a marketing label for a bundle of AI tools. It is a **versioned geometric model of the company, implemented in the form of an Actor Graph and operating, in its mature mode, as a digital twin of the organisation.** When such a model is closed over the real event stream and begins computing its own topology, the company acquires not just a digital control loop, but a true cognitive organ.

The path from Fragmented Company to Autonomous Enterprise is not a “digital transformation” programme. It is a transition to a new form of the firm: not a firm that reacts better to the past, but a firm that can compute and design its own future.

This paper has laid out the architectural logic of that transition. The open questions identified in Section 7—scalability, bootstrapping, calibration, empirical validation, and formal semantics—define the research programme that must follow.

Glossary

Actor Graph (AG)

The language and processing mechanism of the company's geometric model. Every transaction changes the graph itself, making AG simultaneously the medium of computation and the object being changed by that computation.

BGCC — Big Graph of Company Context

A concrete instance of a given organisation's model, expressed in AG. AG is the grammar and the machine; BGCC is the concrete company assembled on that grammar.

Company Brain

The company's cognitive organ. A versioned geometric model of the organisation, implemented as an Actor Graph and operating, in its mature mode, as a digital twin.

Fragmented Company (Phase 1)

The initial state in which data, processes, and decisions are scattered across disconnected systems. No unified regulator exists.

Digital Core (Phase 2)

A unified computational layer implementing the basic regulator factorisation $\pi^* = g \circ \sigma_{\text{prag}}$.

Smart Company (Phase 3)

A company that has built a full geometric model and brought it into digital twin mode, with a computable body of its own.

Autonomous Enterprise (Phase 4)

A state in which the human exits the routine operating loop and remains at the level of goals, policy, and boundaries, defined by the topology of AG.

Geometric model

A computable representation of the company in an engineering sense analogous to CAD assemblies: actors, roles, resources, relationships, states, constraints, and transitions.

Digital Twin (DTO)

A geometric model brought into continuous synchronisation with the company's actual state.

Management debt

The economic shadow of unsuppressed organisational noise. Accrues when coordination work that the regulator should handle is forced onto humans by failures of σ_{prag} .

$$\pi^* = g \circ \sigma_{\text{prag}}$$

The factorisation of a minimal good regulator. σ_{prag} suppresses pragmatically irrelevant distinctions; g selects the action.

$$AG = (\lambda AG. AG(AG))(\lambda AG. AG(AG))$$

The self-computation formula. Captures the architectural closure of the regulator upon its own geometry. In AG, self-application is guarded corecursion: each step is a finite, committed transaction checked against S3 invariants.

E_{acc} The space of acceptable outcomes. Defined by the human; sets the direction of regulation.

Algedonic signals

Alerts that by definition must bypass the normal filters and reach the highest level of governance (Beer).

Hybrid Intelligence

A concrete transaction protocol inside AG in which AI accelerates distinction and the human

holds policy and the right to alter the structure of the admissible.

Meta-regulation (S3 / S4 / S5)

Levels of regulation in Beer's Viable System Model, mapped to concrete AG structures. S3: invariant nodes and guard edges (hard constraints). S4: adaptation subgraph (recalibration, topology proposals). S5: policy nodes (goals, identity, boundaries of the admissible).

References

- [1] Ashby, W.R. *An Introduction to Cybernetics*. London: Chapman & Hall, 1956.
- [2] Ashby, W.R. Requisite variety and its implications for the control of complex systems. *Cybernetica*, 1(2), 1958.
- [3] Barendregt, H. P. *The Lambda Calculus: Its Syntax and Semantics*. Rev. ed. Amsterdam: North-Holland, 1984.
- [4] Barwise, J., Perry, J. *Situations and Attitudes*. Cambridge, MA: MIT Press, 1983.
- [5] Beer, S. *Brain of the Firm: The Managerial Cybernetics of Organization*. London: Allen Lane, 1972. (2nd ed. — Wiley, 1981.)
- [6] Beer, S. *The Heart of Enterprise*. Chichester: Wiley, 1979.
- [7] Beer, S. *Diagnosing the System for Organizations*. Chichester: Wiley, 1985.
- [8] Conant, R. C., Ashby, W. R. Every good regulator of a system must be a model of that system. *International Journal of Systems Science*, 1(2), 1970.
- [9] Cyert, R. M., March, J. G. *A Behavioral Theory of the Firm*. Englewood Cliffs, NJ: Prentice-Hall, 1963.
- [10] Grieves, M., Vickers, J. Digital twin: mitigating unpredictable, undesirable emergent behavior in complex systems. In: *Transdisciplinary Perspectives on Complex Systems*. Springer, 2017.
- [11] Hewitt, C., Bishop, P., Steiger, R. A universal modular ACTOR formalism for artificial intelligence. *Proceedings of IJCAI '73*, 1973.
- [12] Lankhorst, M. et al. *Enterprise Architecture at Work*. 4th ed. Berlin: Springer, 2017. (Archimate 3.x.)
- [13] Milner, R. *Communicating and Mobile Systems: The Pi-Calculus*. Cambridge: Cambridge University Press, 1999.
- [14] Milner, R. *A Calculus of Communicating Systems*. LNCS vol. 92. Berlin: Springer, 1980.
- [15] Sangiorgi, D. *Introduction to Bisimulation and Coinduction*. Cambridge: Cambridge University Press, 2012.
- [16] Scheer, A.-W. *ARIS — Business Process Modeling*. 3rd ed. Berlin: Springer, 2000.
- [17] The Open Group. *TOGAF Standard, Version 9.2*. Van Haren Publishing, 2018.
- [18] Wooldridge, M., Jennings, N. R. Intelligent agents: theory and practice. *The Knowledge Engineering Review*, 10(2), 1995.
- [19] Zachman, J. A. A framework for information systems architecture. *IBM Systems Journal*, 26(3), 1987.
- [20] Vityaz, A. Universal computing element. U.S. Patent Application US20170192795A1, 2017.

- [21] Vityaz, A. Actor graph engine. U.S. Patent Application US20240378338A1, 2024.
- [22] Vityaz, O. A phase model of enterprise evolution: from fragmentation to the autonomous enterprise. Working paper, ResearchGate, 2026. <https://www.researchgate.net/publication/403387171>
- [23] Vityaz, O. Management debt — Part I: concept, metrics, and principles for attributing materialised debts to actor accounts. Working paper, ResearchGate, 2026. <https://www.researchgate.net/publication/402775057>
- [24] Vityaz, O. Active transaction graphs: a formal framework for transactional interactive systems. Working paper, ResearchGate, 2026. <https://www.researchgate.net/publication/401697820>
- [25] Vityaz, O. On the necessity of noise suppression for minimal good regulators: factorisation theorems and a closure conjecture. Working paper, ResearchGate, January 2026. <https://www.researchgate.net/publication/400615896>

Disclosure of Interest

The author is the founder of Corezoid Inc. and the creator of the Simulator / Actor Graph platform. Corezoid and simulator.company, mentioned in the text as an engineering instantiation of the described architecture, are products of the author’s company. This paper presents an architectural position, not an independent product evaluation.

[NotebookLM](#)